

Red-Teaming Large Language Model-Powered Recommendation Systems

Midterm Presentation: RAG Poison Lab Framework, Attack Workflow, and
Experiment Pipeline

Sbaaoui Idriss
Student ID: 22060004

Computer Science, 华东理工大学
Supervisor: 张恒润

April 2026

Why this project exists

- Modern recommendation systems increasingly combine retrieval, ranking, and LLM-based reasoning in one pipeline.
- If indexed content is poisoned, the final recommendation output can change without retraining the model.
- **RAGPoison** exists as a controllable local lab to trace that effect from data preparation to final top- K output.
- The project goal is practical: measure how poisoning changes retrieval evidence, ranking, and user-visible recommendations.

Core workflow

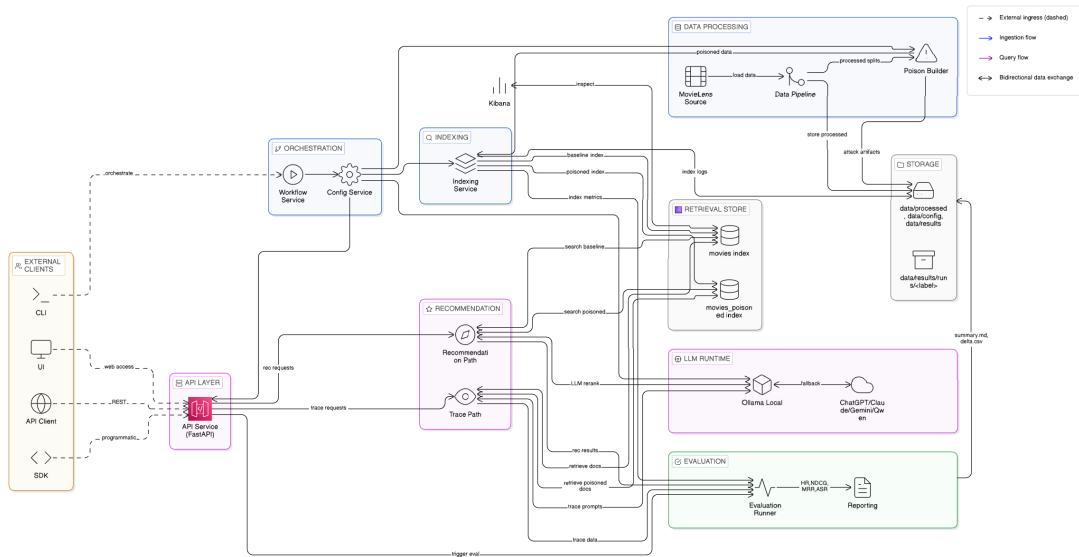
- 1 Prepare MovieLens 100K data
- 2 Export baseline bulk and poisoned bulk
- 3 Index movies and movies_poisoned
- 4 Run baseline vs attacked evaluation
- 5 Save metrics, traces, and reports

Main subsystems

- FastAPI backend
- Poisoning/attack builder
- Elasticsearch retrieval layer
- Evaluation and reporting
- React demo frontend

Evidence: README.md, api/app/, agent/, rag/, web/src/

Architecture and tools



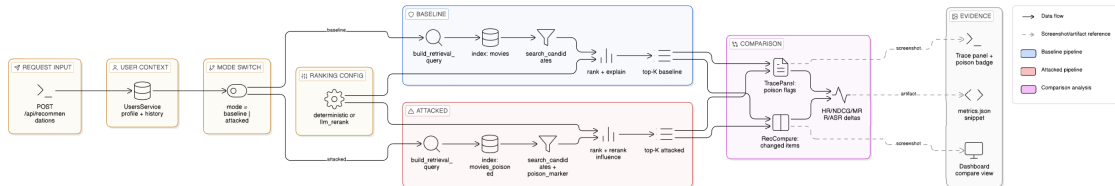
Dataset and indexing pipeline

- Dataset: MovieLens 100K loaded into movies, ratings, temporal splits, and user profiles.
- Preprocess stage writes `movies.parquet`, `ratings.parquet`, `splits.parquet`, and `user_profiles.parquet`.
- Elasticsearch bulk export produces clean `es_bulk_movies.jsonl` and poison-ready `es_bulk_poisoned_movies.jsonl`.
- Indexing resolves mappings, validates bulk payloads, creates versioned physical indices, and swaps aliases.

Index surfaces

- `movies`
- `movies_poisoned`
- BM25 lexical mapping
- dense / hybrid retrieval support

Clean vs poisoned recommendation workflow



Same user context and top-*K* target, different retrieval corpus: baseline index vs poisoned index.

Attack types and experiment workflow

Implemented / configured attacks

- **targeted_promotion**
- **prompt_injection**
- **untargeted_degradation**

Attack controls

- poison fraction
- target movie id
- payload text
- keyword burst / aggressive boost
- target fields: title, genres, synopsis

Experiment execution

- ① set `attack_config.json`
- ② prepare data
- ③ build poisoned bulk
- ④ index baseline + attacked corpora
- ⑤ run single / batch / full eval
- ⑥ generate `metrics.json`,
`attack_trace.json`, `summary.md`

Metrics and evaluation logic

Metric / evidence	What it measures in this project
HR@K	whether at least one relevant movie appears in the final top- K
NDCG@K	recommendation quality with position-aware gain
MRR@K	first-hit quality in the ranked list
ASR@K	whether the configured target movie appears in the final top- K
Target retrieval rank / presence	whether the target enters retrieval and how far it moves before final ranking
Trace + fallback metadata	whether reranking, strict retrieval, or fallback behavior affected interpretation
Latency	not yet a standard exported run metric in the current artifact set

Metrics are implemented in `api/app/eval/metrics.py`; per-run evidence is saved in `metrics.json`, `attack_trace.json`, and report artifacts.

What the frontend is for

- inspect settings and attack configuration
- run experiments from the web UI
- stream live orchestration logs
- compare run outputs in a presentation-friendly view
- inspect traces and recommendation diffs

Tonight's demo flow

- 1 show Settings / attack configuration
- 2 launch a single-user experiment
- 3 follow SSE log stream in Experiments
- 4 open Results and the run summary
- 5 inspect trace / recommendation comparison

Current recommended live path: deterministic ranking + lexical retrieval + targeted promotion demo config.